

Aula 1 - Introdução e instalação de Docker

Docupedia Export

Author:Cardoso Cristian (CtP/ETS)

Date:21-May-2026 20:48

Table of Contents

1 Situação	4
1.1 Hora do pensamento ☐	4
1.1.1 Conclusão	5
2 O que é virtualização?	6
3 Técnicas de virtualização	7
3.1 Em cima de um Sistema Operacional	7
3.2 Diretamente no hardware	8
3.3 Hora do pensamento ☐	10
4 Soluções para conflito de ambientes	11
4.1 Hora do pensamento ☐	12
4.2 Conclusão	12
4.3 Docker e Containers	12
5 Funcionamento de Containers	15
6 Instalando Docker Desktop	17
6.1 Passo 1 configurando o WSL	17
6.2 Passo 2 - Conferir/Ativar Virtualização no seu PC	17
6.3 Passo 3 - Baixar Docker	17
6.4 Passo 4 - Configurando proxy com o Docker	17
7 Iniciando com Docker Desktop	21
7.1 Containers e Imagens	21
7.2 Dockerfile	21
7.2.1 Camadas no Dockerfile	24
7.3 Comandos Dockerfile	28
8 Atividade	31

8.1 Dockerfile

31

1 Situação

Temos quatro aplicações desenvolvidas: Sonar, Cheese,, MusicSync, Skuka e precisamos subir elas para a produção. Abaixo temos as dependências de cada aplicação e o hardware que temos disponível para hospedar todas as aplicações:

Aplicação	Back-end	Front-end	Banco de dados
Sonar	.NET 7.0	Angular 14	PostgreSQL
Cheese	.NET9.0	Angular 17	SQLServer
MusicSync	.NET5.0	Angular 12	MongoDB
Skuka	.NET10.0	Angular 18	MySQL



16gb RAM

i5-13420H

SSD 512GB

Windows 11

1.1 Hora do pensamento ☞

Vai ter alguma complicação para rodar todas as aplicações e mante-las? Se sim quais? Se não, por quê?

1.1.1 Conclusão

Sim, teremos diversos problemas.

- Conflito entre versões: Toda atualização de biblioteca de uma aplicação corre o risco de quebrar outra,
- Conflito de Portas: Bancos de dados e serviços web vão brigar pelas mesmas portas de rede padrão.
- Manutenção: Caso seja necessário reiniciar o servidor para aplicar uma alteração em qualquer aplicação vai derrubar todas as outras.

2 O que é virtualização?

Virtualização é ter vários computadores independentes em um único computador físico.



3 Técnicas de virtualização

Existem duas principais técnicas de virtualização:

3.1 Em cima de um Sistema Operacional

Quando seu hardware tem primeiramente um sistema operacional já iniciado (Windows, Linux, MacOS). E então com um virtualizador instalado, como o [Oracle VirtualBox](#), ele irá pedir permissão para o sistema operacional antes de usar o hardware para podermos definir imagens de sistemas operacionais, onde definimos quanto do nosso hardware principal ela poderá utilizar para seu processamento:

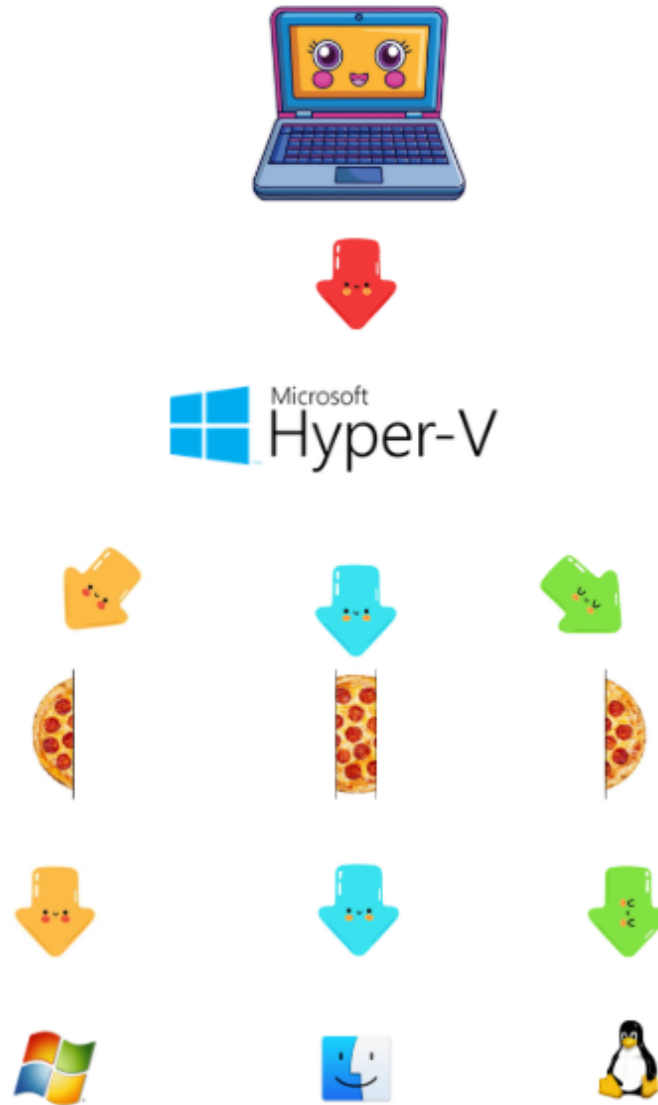




3.2 Diretamente no hardware

diferente da VirtualBox, onde o fluxo era Hardware → OS → Virtualizador → Máquina virtual, agora o virtualizador é o "Chefe" do hardware, ele é o primeiro sistema que liga no computador, o **Hypervisor**, e particiona os recursos para as máquinas virtuais. Como se o computador tivesse sido feito só para criar VMs.

****Não é equivalente a um Dual Boot pois neste caso temos todos os sistemas operacionais rodando simultaneamente.**

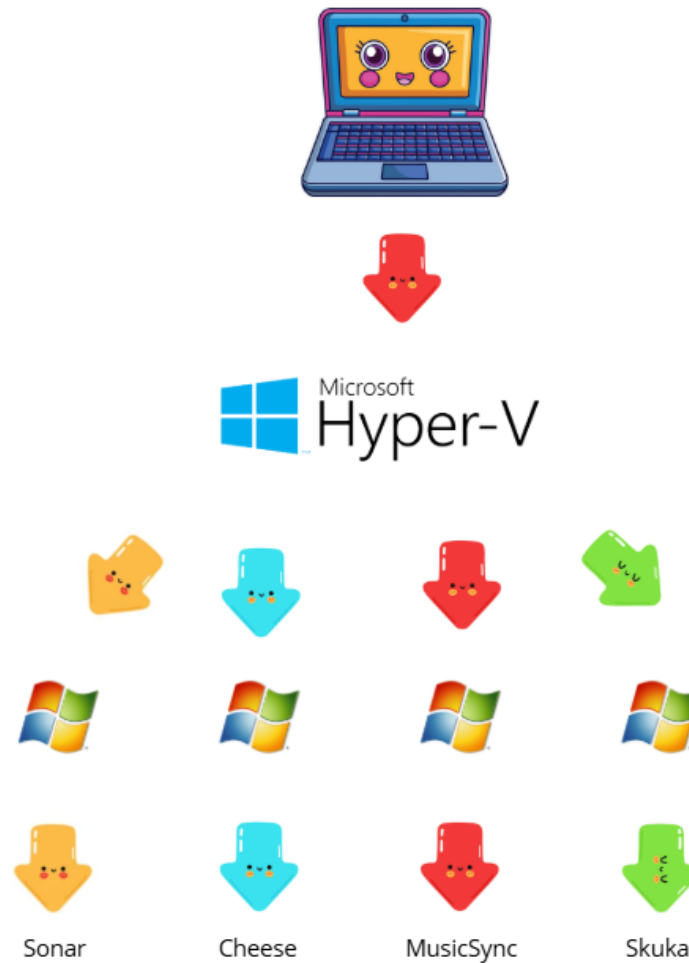


3.3 Hora do pensamento

E agora, é possível resolver o problema anterior?

4 Soluções para conflito de ambientes

Agora que sabemos que é possível criar vários ambientes em um computador fica fácil certo? Podemos criar um ambiente quaisquer e colocar cada aplicação para rodar, como por exemplo, 4 linux rodando com hypervisor diretamente no hardware, sem conflitos de dependências.



4.1 Hora do pensamento ☐

O que acham desta maneira?

4.2 Conclusão

Fazendo deste modo temos um sistema operacional inteiro destinado a apenas uma única aplicação, temos interface gráfica, serviços de usuário, recursos copilot, e outros diversos que não agregar em nada a uma aplicação.

Além de que:

- O windows é um dos sistemas operacionais com maior custo operacional nos dias de hoje
- Boa parte do poder de processamento é perdido em serviços voltados ao uso humano, não a servidores
- Atualizações automáticas e serviços em segundo plano introduzem risco operacional e indisponibilidade inesperada
- O ambiente se torna caro de manter, difícil de reproduzir e pouco previsível

Isso se aplica tanto para Windows, Linux MacOS, entre outros, a única diferença é o tamanho do estrago, alguns sistemas são menos robustos que outros, Mas o problema persiste, um sistema operacional inteiro destinado a uma única aplicação é desperdício. independente do sistema usado.

4.3 Docker e Containers

Docker é um sistema de gerenciamento de containers, que depende de um Kernel Linux, cada container roda uma única aplicação, com um ambiente minimo para a aplicação, sem sistema operacional completo, apenas um ambiente único,

****Ambiente:** É o conjunto de dependências necessárias para manter a aplicação:

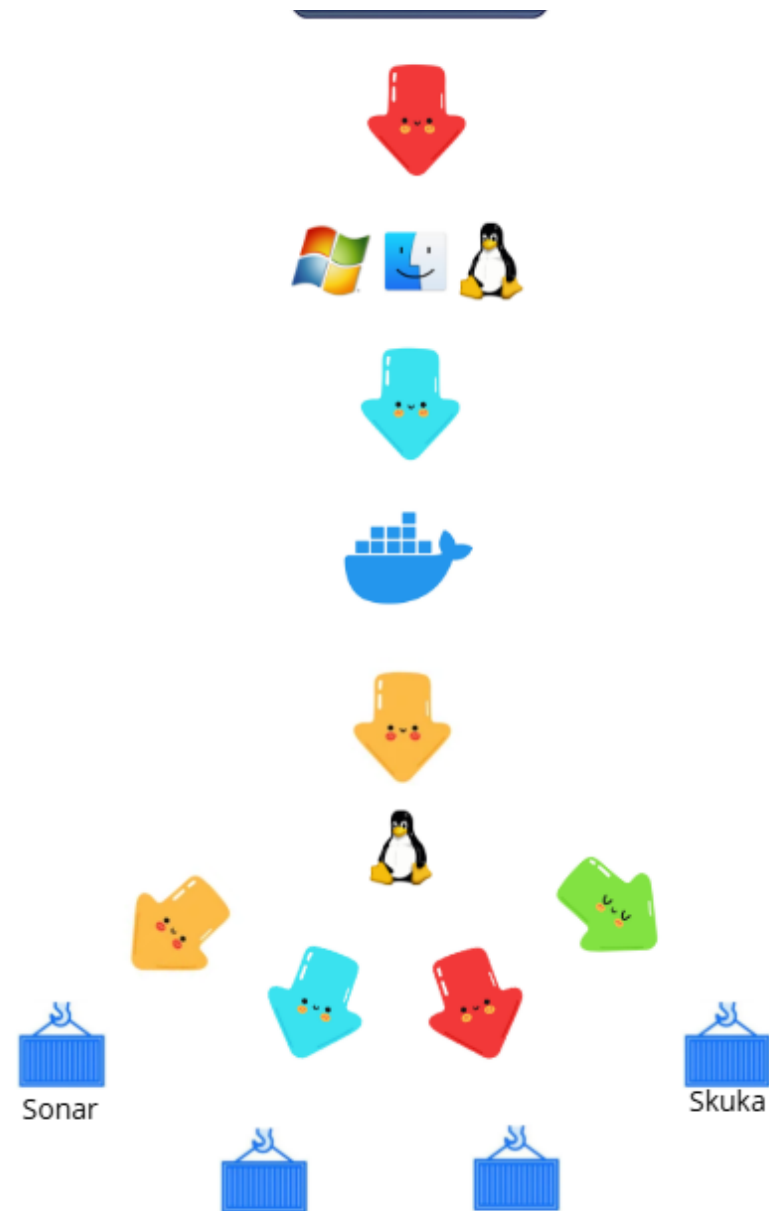
- Bibliotecas
- Runtime (ex: .NET, Node, Java)
- Variáveis de ambiente
- Estrutura de arquivos necessária

****Linux:** Não é um sistema operacional utilizável por usuários comuns, é um kernel, o Linux é a base para desenvolver distribuições, como por exemplo: ubuntu, arch, debian, popos.

- Kernel Linux == motor do carro
- Distribuição Linux == carro completo (volante, bancos, painel)

Agora utilizando Docker podemos resolver nosso problema inicial:





Cheese

MusicSync

5 Funcionamento de Containers

O Docker cria para cada container um novo processo isolado, definindo um novo diretório root (/) para ele, utilizando namespaces e cgroups para isolamento de processos e recursos, que é um mecanismo do Linux que permite rodar um processo “travado” a uma pasta, fazendo essa pasta parecer ser o root (/) do sistema para aquele processo.

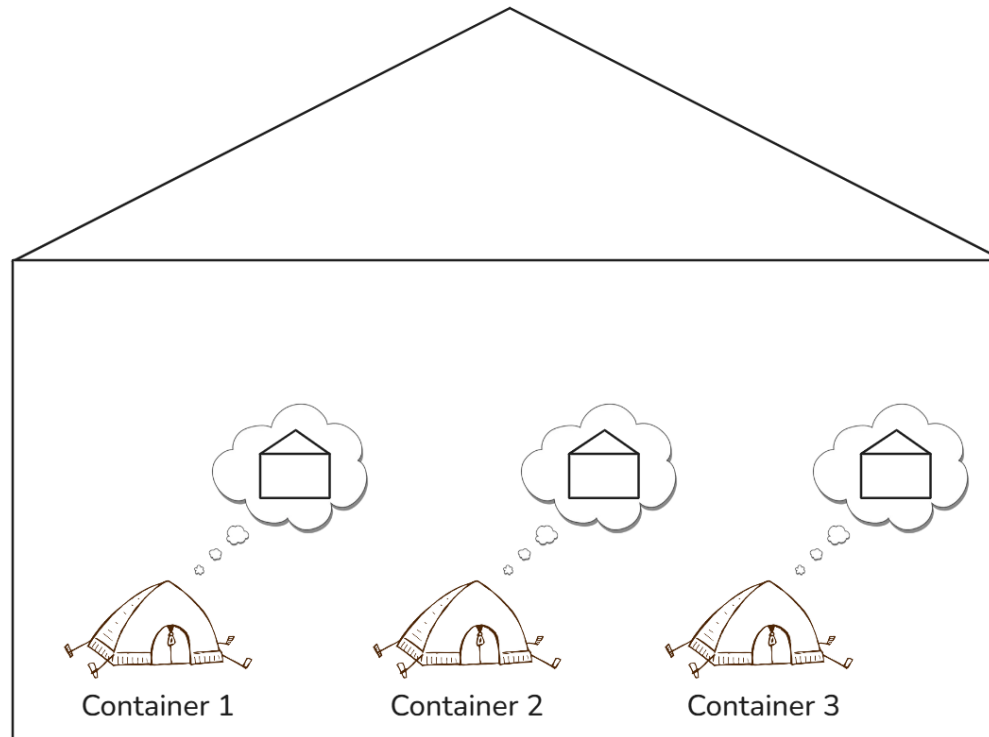
Fora disso, o sistema continua o mesmo.

Imagine que o container seja apenas uma pasta em `var/lib/docker/containers/[nome]/` (exemplo irreal). Para o container, essa pasta `/[nome]/` se torna o seu root (/) absoluto.

Devido a essa troca, o container não consegue acessar o sistema de arquivos do host (o verdadeiro "/") nem de outros containers. Ele vive em sua própria bolha.

Então operações padrão como definir variáveis de ambiente, acessar bibliotecas e executar binários funcionam naturalmente dentro desse contexto isolado, sem ter conflitos com o servidor.

É importante notar a diferença para a virtualização: embora o Docker isole os arquivos e processos, todos os containers compartilham o mesmo Kernel Linux do host, por isso são extremamente leves e eficientes, já que não precisa de um OS inteiro para cada aplicação, além de que o OS usado para gerenciamento não é feito para pessoas usarem no dia a dia, e sim para servidores de fato, mas exige configurações de rede caso você queira comunicação entre os containers.



6 Instalando Docker Desktop

6.1 Passo 1 configurando o WSL

- Instale [WSL clicando aqui](#)
- No terminal como administrador execute:

```
wsl --update
```

6.2 Passo 2 - Conferir/Ativar Virtualização no seu PC

Conferir se a virtualização está ativada no seu PC

- CTRL+SHIFT+ESC (gerenciador de tarefas)
- Performance
- CPU
- Virtualization: enable

Caso não esteja, chame o instrutor para realizar a instalação:

- CTRL+X → A
- Execute:

```
wsl --set-default-version 2
```

- Reinicie o computador

6.3 Passo 3 - Baixar Docker

- [Clique aqui](#)

6.4 Passo 4 - Configurando proxy com o Docker

- **IMPORTANT!**

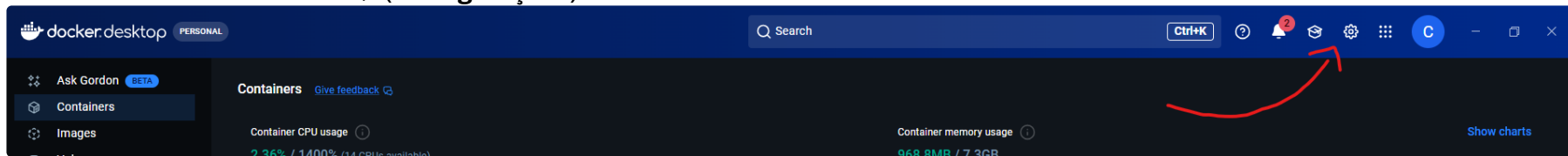
Sempre antes de iniciar o Docker inicie o **RB Local Proxy Manager**. Aperte a tecla WINDOWS e pesquise por **PX by BD** e aperte ENTER.

- Execute o arquivo **Docker Desktop Installer.exe**
- Abra o diretório: *C:/ProgramData/DockerDesktop/*
- Crie uma pasta com o nome */config/*

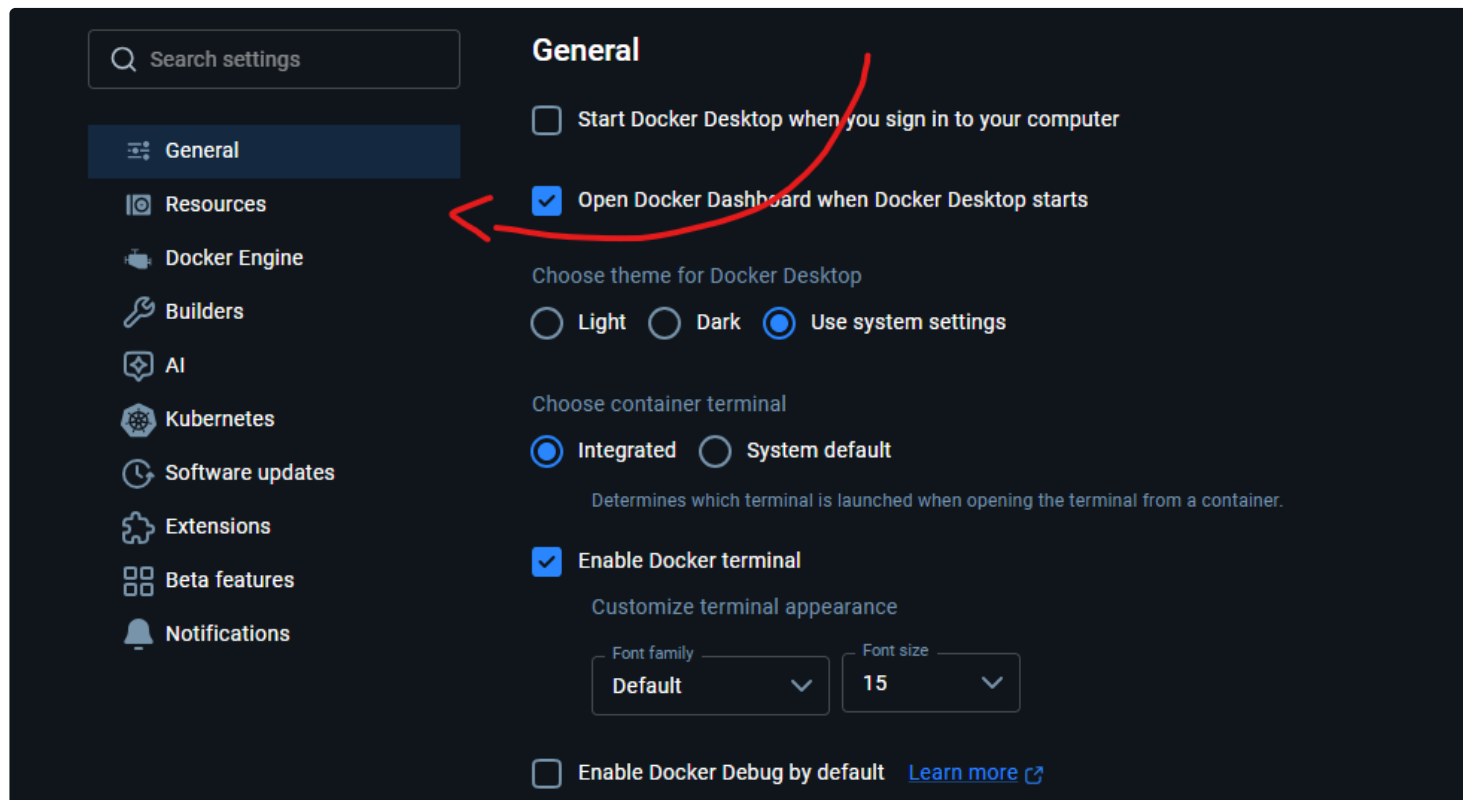
- Dentro da pasta config crie o arquivo *daemon.json*
- Cole dentro do arquivo *daemon.json* o seguinte conteúdo:

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  },
  "dns": [
    "8.8.8.8",
    "1.1.1.1"
  ],
  "experimental": false
}
```

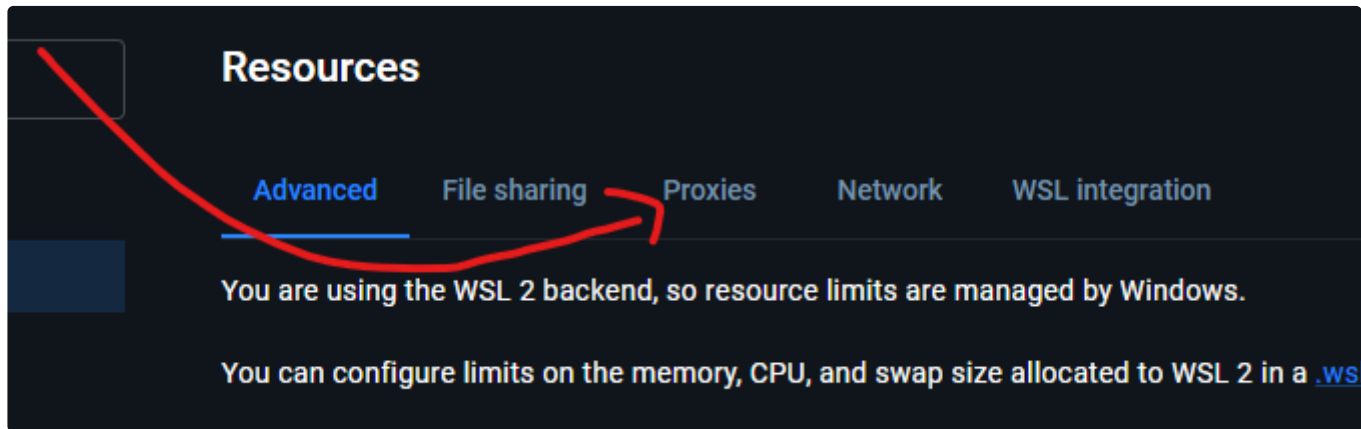
- Dentro do Docker vá em ⚙ (Configurações)



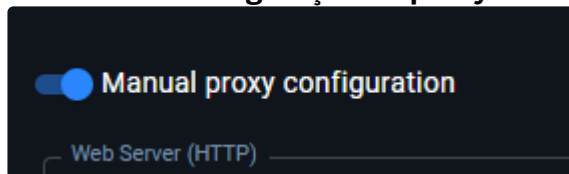
- Dentro das configurações acesse **Resources**



- Dentro de Resources acesse Proxies:



- Ative a **configuração de proxy manual**:



- Configure da seguinte forma:

Web Server (HTTP) & Secure Web Server (HTTPS):

http://localhost:3128

Bypass proxy settings for these hosts & domains:

localhost,127.0.0.1,192.168.,.local,172.*,*.bosch.*,10.4.103.143*

7 Iniciando com Docker Desktop

7.1 Containers e Imagens

Container é como uma máquina virtual descartável, mas como vimos anteriormente ele não é de fato uma máquina virtual, ele utiliza do Kernel Linux (no windows utiliza WSL) para funcionar, então na realidade ele cria um ambiente dentro do nosso Kernel, mas esse ambiente não surge do zero, por isso temos as imagens.

Imagem é o que fornece o sistema de arquivos, bibliotecas e comandos básicos que o container vai usar.

A **imagem** é como uma forma de biscoitos e o **container** um único biscoito, podemos usar o molde para fazer vários biscoitos.

7.2 Dockerfile

Entretanto podemos fazer configurações em cima de uma imagem escolhida, então definimos a estrutura que nosso container terá baseado na imagem definida, e após isso fazemos demais configurações, para isso precisaremos de um Dockerfile.

Vamos entender melhor fazendo, crie um arquivo "Dockerfile", sem extensão.

Com isso aplicamos a imagem mssql/server:

```
FROM mcr.microsoft.com/mssql/server:2022-latest
```

```
ENV ACCEPT_EULA=Y  
ENV MSSQL_PID=Developer
```

ACCEPT_EULA - define se voce aceita ou não o contrato de licença, funciona contanto que o software identifique essa variável, no caso SQLSERVER identifica.
MSSQL_PID - define o tipo de licença do SQLSERVER que vai rodar no container. Developer é a edição gratuita

```
ENV MSSQL_SA_PASSWORD=821hs91ukd_uwq&1*9j
```

Significa que você está definindo a senha do usuário **sa** (administrador) do SQL Server dentro do container.

Detalhes importantes:

- Essa variável é usada pelo SQLSERVER durante a inicialização para configurar a conta sa.
- O valor deve atender aos requisitos de complexidade do SQLSERVER (mínimo 8 caracteres, mistura de letras, números e símbolos)

```
EXPOSE 1433
```

EXPOSE - Define que iremos expor através da porta 1433, porta essa que o SQLSERVER está escutando por padrão.

Dockerfile final:

```
FROM mcr.microsoft.com/mssql/server:2022-latest

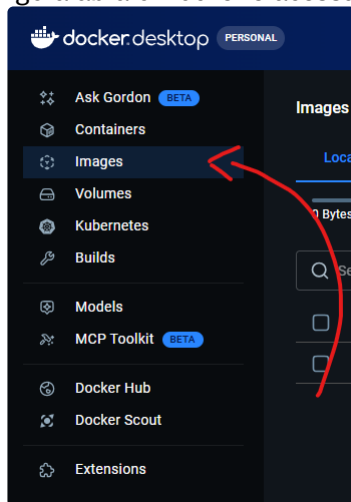
ENV ACCEPT_EULA=Y
ENV MSSQL_PID=Developer
ENV MSSQL_SA_PASSWORD=821hs91ukd_uwq&1*9j

EXPOSE 1433
```

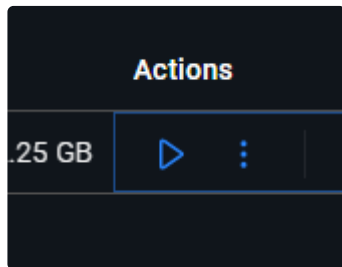
Agora para utilizarmos esse dockerfile para criar uma imagem no nosso DockerDesktop precisamos rodar o seguinte comando:

```
Docker build -t <NOME>:latest ./
```

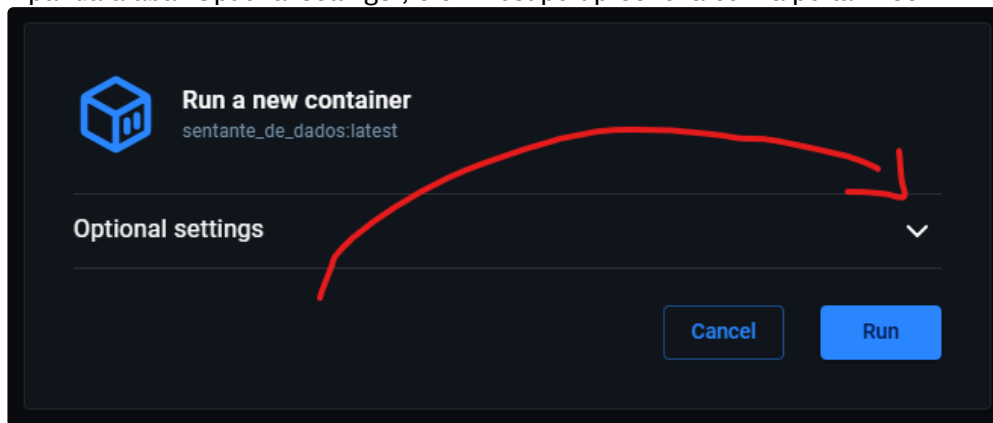
Agora abra o Docker e acessa a aba de "Imagens"

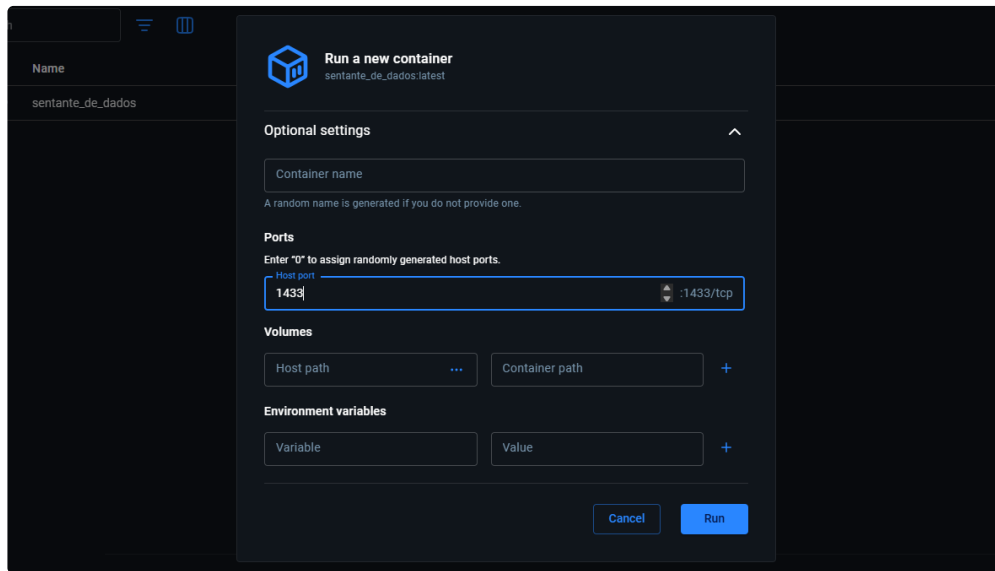


Em actions inicie a sua imagem, isso vai criar um novo container na aba "Containers".



Expanda a aba "Optional settings", e em host port preencha com a porta 1433





- E então aperte em **Run**

Abra o **SQL Server Management Studio XX**

- **Server name:** localhost,1433
- **Authentication:** SQL Server Authentication
- **Login:** sa
- **Password:** 821hs91ukd_uwq&1*9j

(porta exposta pelo seu container, definida no Dockerfile)

(administrador do sql)

(senha que foi definida no Dockerfile)

****Marque "[X] Trust server certificate"**

Você acaba de se conectar com seu container! 🎉

7.2.1 Camadas no Dockerfile

Toda linha no Dockerfile gerará uma nova "camada", cada camada carrega com si uma etapa da configuração desta imagem:

```
FROM mcr.microsoft.com/mssql/server:2022-latest | CAMADA 1
ENV ACCEPT_EULA=Y | CAMADA 2
```



```
ENV MSSQL_PID=Developer          | CAMADA 3
ENV MSSQL_SA_PASSWORD=821hs91ukd_uwq&1*9j | CAMADA 4

EXPOSE 1433                      | CAMADA 5
```

Cada camada carrega uma parte da configuração da imagem, quando utilizamos o "FROM xxxxxx", estamos baixando várias camadas já configuradas para aplicarmos no nosso Dockerfile,

Para melhor entendimento podemos executar o seguinte comando e ver como o Docker lidou com nosso Dockerfile:

Docker **history** <NOME_DA_SUA_IMAGEM>

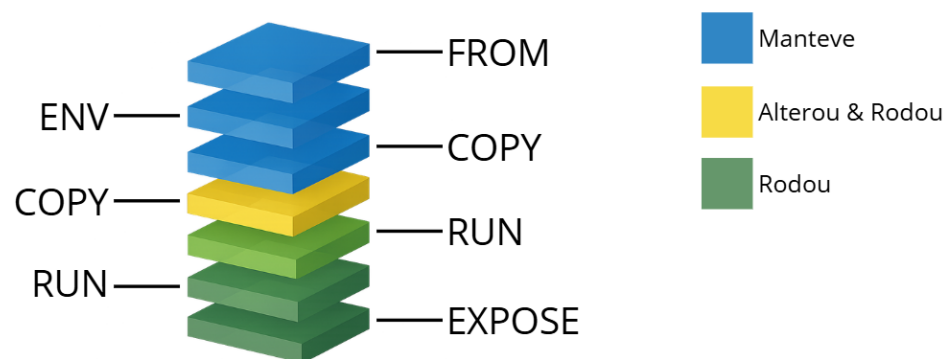
IMAGE	CREATED	CREATED BY	SIZE	COMMENT
a4cfd1807007	2 weeks ago	EXPOSE [1433/tcp]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV MSSQL_SA_PASSWORD=821hs91ukd_uwq&1*9j	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV MSSQL_PID=Developer	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV ACCEPT_EULA=Y	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	CMD ["/opt/mssql/bin/sqlservr"]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENTRYPOINT ["/opt/mssql/bin/launch_sqlservr....	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	USER mssql	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN 1 ARG_MSSQL_PID=developer /bin/sh -c EX...	196MB	buildkit.dockerfile.v0

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
<missing>	2 weeks ago	RUN 1 ARG_MSSQL_PID=developer /bin/sh -c ta...	1.37GB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV MSSQL_PID=developer	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV CONFIG_EDGE_BUILD=	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV MSSQL_RPC_PORT=135	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	EXPOSE [1433/tcp]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	LABEL vendor=Microsoft com.microsoft.product...	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ARG ARG_MSSQL_PID=developer	0B	buildkit.dockerfile.v0
<missing>	4 months ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B	
<missing>	4 months ago	/bin/sh -c #(nop) ADD file:d025507456f1d7d19...	87.5MB	
<missing>	4 months ago	/bin/sh -c #(nop) LABEL org.opencontainers....	0B	
<missing>	4 months ago	/bin/sh -c #(nop) LABEL org.opencontainers....	0B	
<missing>	4 months ago	/bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH	0B	

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
<missing>	4 months ago	/bin/sh -c #(nop) ARG RELEASE	0B	

IMPORTANTE !

Agora um detalhe que é o que faz as camadas serem tão úteis. As camadas são imutáveis, portando depois que a imagem subir a única forma de modificar algo é fazendo build novamente, as camadas funcionam como uma pilha, e a imagem se aproveita disso para otimizar quando houver mudanças na sua configuração (novo build). Se por exemplo no nosso caso quisermos alterar a senha do nosso banco de dados, na hora do build não vamos precisar rodar o Dockerfile inteiro, uma vez que o docker utiliza de cache para configuração, então ele vai ver a sua nova configuração e comparar com a antiga, se ele ver que não houve mudanças ele nem executa a camada, só executa a partir do momento que encontrar uma mudança, portanto se alterarmos a senha não vamos executar o "FROM" que como pudemos ver é uma execução pesada, só vamos precisar executar o "ENV", execução leve



```
FROM mcr.microsoft.com/mssql/server:2022-latest
```

<--- NÃO ALTEROU, PULA

```
ENV ACCEPT_EULA=Y
```

<--- NÃO ALTEROU, PULA

```
ENV MSSQL_PID=Developer
```

<--- NÃO ALTEROU, PULA

```
ENV MSSQL_SA_PASSWORD=n0op0qaxgk103
```

<--- ALTEROU, EXECUTA ISSO E TUDO QUE VEM ABAIXO

```
EXPOSE 1433
```

IMPORTANTE 2!

Isso também se aplica a diferentes imagens, se criarmos um novo Dockerfile com a camada "FROM mcr.microsoft.com/mssql/server:2022-latest", não teremos que baixar novamente pois temos isso no nosso cache. O mais legal é que isso não é apenas para essa camada mas sim para todas, contanto que o comando seja o mesmo ele vai estar no nosso cache e não vamos precisar fazer a parte pesada dele, vamos apenas fazer uma cópia instantânea do conteúdo necessário.

7.3 Comandos Dockerfile

FROM <image>

Importa configurações de camadas já definidas

FROM <image> **AS** <NAME>

Importa configurações de camadas já definidas e define um alias para se referir a todo o bloco que virá na parte de baixo

WORKDIR <path>

Define a pasta de trabalho, onde o docker vai olhar para suas execuções. Não precisa ser um caminho que já exista, se não existir ele criará.

COPY <host_dir_path> <container_dir_path>

Copia arquivos da sua máquina para dentro das pastas do container

RUN <command>

Comando que vai rodar no terminal padrão. É usado pra instalar pacotes, compilar código, etc

ENV <name>=<value>

Define variáveis de ambiente que ficam disponíveis durante o build e quando o container estiver rodando também

ARG <name>=<value>

Define argumentos de build disponíveis durante o processo de build da imagem.

EXPOSE <port>

Aponta para qual porta o container vai "ouvir".

ENTRYPOINT [<text>, ...]

Algo que vai ser executado no terminal quando o container for iniciado. Caso algum parametro adicional seja colocado ele vai ser colocado depois dos comandos definidos no entrypoint

exemplo:

```
ENTRYPOINT ["echo", "Are", "you", "ready"]
```

output:

```
Docker run <image>
>> Are you ready

Docker run <image> Kids?
>> Are you ready Kids?
```

CMD [<string>, ...]

Algo que vai ser executado caso o container tenha sido executado sem argumentos adicionais.

exemplo:

```
CMD ["echo", "Kids"]
```

output:

```
Docker run <image>
>> Kids

Docker run <image> Girls
>> docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime create failed:
runc create failed: unable to
>> start container process: error during container init: exec: "Girls": executable file not found in $PATH
>>
>> Run 'docker run --help' for more information
```

Esse erro ocorre porque sobrescrevemos o CMD inteiro, incluindo o "echo", portanto executamos apenas "Girls" no terminal, ou seja, nosso sistema tentou encontrar um comando chamado "Girls" e não encontrou.

Pra resolver isso podemos fazer uma **fusão**!



```
ENTRYPOINT ["echo", "Are", "you", "ready"]  
CMD ["Kids?"]
```

output:

```
Docker run <image>  
>> Are you ready Kids?  
  
Docker run <image> Girls?  
>> Are you ready Girls?
```

8 Atividade

Monte um Dockerfile para uma API em .NET 9 com um único endpoint `"/docker"` que retorna a string `"Banguela"`



8.1 Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:9.0

WORKDIR /app

ENV HTTP_PROXY="host.docker.internal:3128"
ENV HTTPS_PROXY="host.docker.internal:3128"

COPY ./API/*.csproj ./
RUN dotnet restore
COPY ./API ./

CMD ["dotnet", "run", "--urls=http://0.0.0.0:1234"]

EXPOSE 1234
```

Passo a passo:

- Importa a pré-configuração oficial da microsoft
- Cria e entra na pasta onde trabalharemos dentro do nosso container
- Define proxy http
- Define proxy https
- Copia apenas o *csproj*, isso é uma boa prática devido a estrutura de camadas explicada anteriormente. Uma vez que buildamos muitas vezes, se copiarmos tudo da API primeiro e depois dermos *dotnet restore*, precisaremos baixar tudo todas as vezes, mas se copiarmos apenas o *.csproj*, ele vai identificar que os pacotes não foram alterados e não vai precisar baixar.
- Baixa os pacotes
- Copia os outros arquivos para dentro do nosso container
- Define que quando rodarmos o container, caso nada seja escrito ao lado, rodar "dotnet run --urls=http://0.0.0.0:1234"
- Avisa que seu container está configurado para expor sua porta "1234"

Podemos adicionar um *.dockerignore* para garantir que não será copiado para dentro do nosso container coisas como */obj*, */bin* e etc.

```
/obj  
/bin  
  
.dockerignore
```

Estrutura final:

